

PaperTrail

Reads the fine print so you don't have to.

A complete, beginner-friendly project guide
and full code walkthrough

Next.js • Express • FastAPI • MySQL • MongoDB • Qdrant • Redis • Groq • Docker

What's inside this guide

- 1. What is PaperTrail? (the problem and the idea)
- 2. The big picture in one flow
- 3. Why so many technologies? (the tech stack)
- 4. Why FOUR databases?
- 5. The core AI ideas, explained simply
- 6. The project structure
- 7. Infrastructure: Docker Compose
- 8. The Backend (Node + Express + Prisma + BullMQ)
- 9. The Pipeline (Python + FastAPI + AI)
- 10. The Frontend (Next.js + Tailwind)
- 11. Automation: GitHub Actions (CI + cron)
- 12. Why we did not deploy (yet)
- 13. Interview questions and answers
- 14. Limitations and future improvements

1. What is PaperTrail?

The problem

Every day we click 'I agree' on Terms of Service, privacy policies, pricing pages, rental rules and government notices. This 'fine print' quietly rules our lives. It changes all the time, and almost nobody re-reads it. A company can start selling your data, raise a fee, or remove a promise, and you would never notice.

The idea

PaperTrail is like a patient friend who reads that boring fine print for you. You give it a link to watch and a short note about yourself ("I am a renter", "I use Stripe"). PaperTrail reads the page, and every time you check it, it does one of three things:

- **First look:** it gives you a plain-English SUMMARY of the key points.
- **Nothing changed:** it says so calmly.
- **Something changed:** it tells you exactly WHAT changed and HOW it affects YOU.

The clever part is that it detects **meaning** changes, not just word changes. If a sentence is reworded but still means the same thing, PaperTrail stays quiet. If the meaning actually shifts (for example 'we will never sell your data' becomes 'we may sell your data'), it raises the alarm. This is called a **semantic diff**, and it is the heart of the project.

2. The big picture in one flow

Here is the whole journey of a single 'Check for changes' click, from your browser to the answer on your screen:

```
You add a URL -> Backend saves it to MySQL
You click 'Check' -> Pipeline fetches the page
                        | - saves the raw HTML snapshot (Mongoose)
                        | - splits it into clauses (BeautifulSoup)
                        | - turns each clause into an embedding (sentence-transformers)
                        | - stores the embeddings (Odrant)
                        | - compares to the previous version (semantic diff)
                        | - writes a summary / impact (Grok LLM)
Result on screen -> 'Key points' / 'No changes' / 'Change detected'
```

Every technology in the stack has a real, distinct job in that flow. Nothing is there just for show.

3. Why so many technologies?

Each tool below does one job it is genuinely the best at. This is called using the 'right tool for the right job'.

Technology	Its job in PaperTrail
Next.js + React + TypeScript + Tailwind	The dashboard you look at (the frontend).
Node.js + Express	The user-facing REST API (users, pages, profiles, crawl).
MySQL + Prisma	Neat, structured data: users, watched pages, profiles.
MongoDB	Big messy raw page snapshots (every version).
Redis + BullMQ	A background job queue for the crawls.
Python + FastAPI	The AI pipeline: fetch, clean, split, embed, diff, explain.

sentence-transformers	Local, free 'meaning fingerprints' (embeddings).
Qdrant	A vector database that powers the semantic diff.
Groq (Llama 3.3)	A free LLM that writes summaries and impact.
Docker Compose	Runs all four databases together, locally, for free.
Git + GitHub + Actions	Version control, CI, and a free scheduled cron.

4. Why FOUR databases?

Beginners often ask: why not just one database? Because the data has four very different shapes, and each store is built for a different shape.

MySQL (with Prisma) - the tidy spreadsheet

Users, profiles and the watch-list are neat rows and columns with clear relationships (one user has many pages). A relational database like MySQL is perfect for that.

MongoDB - the box of folders

A fetched web page is a big, messy blob of HTML of unknown size and shape. MongoDB is a document store, great for dumping whole documents. Each time we fetch a page we drop a new 'folder' (snapshot) in the box, keeping every version so we can compare later.

Qdrant - the map of meaning

To compare pages by MEANING, we turn each clause into a list of numbers (an embedding) and store it in a vector database. Qdrant is built to answer 'what is nearest to this point?', which is exactly 'what means almost the same?'. Normal databases cannot do that.

Redis - the super-fast notebook

Redis keeps data in memory, so it is lightning fast. We use it as the backbone of the job queue (BullMQ stores its 'tickets' in Redis).

5. The core AI ideas, explained simply

Embeddings - meaning as numbers

Imagine a giant map of meaning. Every sentence gets a pin on it. Sentences that mean similar things get pinned close together. An **embedding** is just the coordinates of a sentence's pin, written as a list of numbers (ours are 384 numbers long). A small AI model reads text and returns its coordinates. Once everything is coordinates, 'do these two clauses mean the same?' becomes simple geometry: are their pins close?

Vector search - finding nearby meaning

A vector database stores those pins and can instantly find the nearest one to any point. That is how we ask, for a new clause, 'is there an old clause that means almost the same?'.

Semantic diff - the heart

For each clause in the NEW version we ask Qdrant for the nearest clause in the OLD version. If a very close neighbour exists (closeness above a threshold of 0.85), the clause is basically unchanged, even if reworded, so we ignore it. If there is NO close neighbour, the clause is genuinely new or changed, so we flag it. We do the reverse to find removed clauses. This is why a reworded sentence stays quiet but a real meaning change is caught.

Job queue - work without freezing

Slow work (fetching, embedding) should not make the app freeze. A job queue lets us drop 'check this page' tickets into a line instantly; a background worker processes them one by one. BullMQ is the

queue; Redis stores the tickets.

6. The project structure

PaperTrail is a 'monorepo': three little programs living in one repository, each with its own room.

```
nanertrail/  
  docker-compose.yml # MySQL, MongoDB, Odrant, Redis  
  .github/workflows/  
    ci.yml # build/test-check on every push  
    scheduled-crawl.yml # daily cron to trigger crawls  
  backend/ # Node + Express + Prisma + BullMQ  
    prisma/schema.prisma # User, Profile, WatchedSource  
    src/index.ts # the API routes  
    src/worker.ts # the background worker  
    src/lib/{prisma,redis,queue}.ts  
  pipeline/ # Python + FastAPI (the AI brain)  
    app/main.py # all the endpoints  
    app/db.py # MongoDB connection  
    app/processing.py # HTML -> clean text -> clauses  
    app/embeddings.py # local embedding model  
    app/vectors.py # Odrant + semantic diff  
    app/explain.py # Groq summary and explanation  
  frontend/ # Next.js dashboard  
    src/app/page.tsx # the whole dashboard UI  
  demo/policy.html # editable page to test change detection
```

7. Infrastructure: Docker Compose

Docker runs each database inside a sealed 'lunchbox' (a container) so it works the same on any computer, with no messy installs. Docker Compose is a menu card listing all the lunchboxes so we start them together with one command: **docker compose up -d**.

File: *docker-compose.yml*

```
services:
  mysql:
    image: mysql:8.0
    container_name: papertrail-mysql
    restart: unless-stopped
    env_file:
      - .env
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  mongo:
    image: mongo:7
    container_name: papertrail-mongo
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}
    ports:
      - "27018:27017"
    volumes:
      - mongo_data:/data/db

  qdrant:
    image: qdrant/qdrant:latest
    container_name: papertrail-qdrant
    restart: unless-stopped
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - qdrant_data:/qdrant/storage

  redis:
    image: redis:7
    container_name: papertrail-redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

volumes:
  mysql_data:
  mongo_data:
  qdrant_data:
  redis_data:
```

- **services:** the four databases (mysql, mongo, qdrant, redis).
- **image:** the recipe for each container (e.g. mysql:8.0).
- **ports:** 'left:right' maps a door on your PC to a door inside the container. Note MongoDB uses 27018 on the PC because a native MongoDB was already using 27017.
- **volumes:** storage lockers so data survives even if a container is deleted.
- **environment / env_file:** passwords passed in from the .env file (kept out of Git).

8. The Backend (Node + Express + Prisma + BullMQ)

The backend is the user-facing API. It remembers users, their watched pages and profiles (in MySQL through Prisma), and it queues background crawl jobs (through BullMQ).

8.1 The database shape: `schema.prisma`

Prisma is a translator between our code and MySQL. We describe our tables in one simple file and Prisma builds them and gives us easy functions.

File: `backend/prisma/schema.prisma`

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql"
  url = env("DATABASE_URL")
}

model User {
  id Int @id @default(autoincrement())
  email String @unique
  createdAt DateTime @default(now())

  profile Profile?
  sources WatchedSource[]
}

model Profile {
  id Int @id @default(autoincrement())
  description String @db.Text
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  userId Int @unique
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

model WatchedSource {
  id Int @id @default(autoincrement())
  url String @db.VarChar(2048)
  label String
  createdAt DateTime @default(now())

  userId Int
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

- **model User / Profile / WatchedSource** = three tables.
- **@id @default(autoincrement())** = an automatic unique id per row.
- **@unique** on email = no two users share an email.
- **@relation(... onDelete: Cascade)** = the wire linking tables; deleting a user auto-deletes their profile and pages (no orphans).
- **Profile? and WatchedSource[]** = a user has one optional profile and many pages.

8.2 One shared Prisma client: `lib/prisma.ts`

We create the Prisma client once and reuse it everywhere, instead of making a new connection in every file.

File: `backend/src/lib/prisma.ts`

```
import { PrismaClient } from "@prisma/client";

// One shared Prisma client for the whole app.
// Other files import this instead of making their own.
export const prisma = new PrismaClient();
```

8.3 The API routes: index.ts

This is the whole Express server. It opens a door (port 4000) and answers requests. Below is the complete file, then an explanation of each route.

File: backend/src/index.ts

```
import cors from "cors";
import express from "express";
import { prisma } from "../lib/prisma";
import { crawlQueue } from "../lib/queue";

const app = express();
app.use(cors());
const PORT = process.env.PORT || 4000;
const PIPELINE_URL = process.env.PIPELINE_URL ?? "http://localhost:8000";

app.use(express.json());

// Health check – is the server alive?
app.get("/health", (_req, res) => {
  res.json({ status: "ok", service: "papertrail-backend" });
});

// Home – friendly hello.
app.get("/", (_req, res) => {
  res.send("PaperTrail backend is alive! 🟢");
});

// CREATE a user: POST /users with body { "email": "..."}
app.post("/users", async (req, res) => {
  const { email } = req.body;

  if (!email || typeof email !== "string") {
    return res.status(400).json({ error: "email (string) is required" });
  }

  try {
    const user = await prisma.user.create({ data: { email } });
    return res.status(201).json(user);
  } catch (err) {
    return res
      .status(409)
      .json({ error: "Could not create user – email may already exist" });
  }
});

// LIST all users: GET /users
app.get("/users", async (_req, res) => {
  const users = await prisma.user.findMany();
  return res.json(users);
});

// CREATE a watched source (no duplicate URL or label per user).
app.post("/users/:userId/sources", async (req, res) => {
  const userId = Number(req.params.userId);
  const { url, label } = req.body;

  if (Number.isNaN(userId)) {
    return res.status(400).json({ error: "userId must be a number" });
  }

  if (!url || typeof url !== "string" || !label || typeof label !== "string") {
    return res
      .status(400)
      .json({ error: "url (string) and label (string) are required" });
  }

  // No duplicate URL for this user.
  const dupUrl = await prisma.watchedSource.findFirst({ where: { userId, url } });
  if (dupUrl) {
    return res.status(409).json({ error: "You're already watching this URL." });
  }

  // No duplicate label for this user.
  const dupLabel = await prisma.watchedSource.findFirst({
    where: { userId, label },
  });
  if (dupLabel) {
    return res
      .status(409)
      .json({ error: "You already have a page with that label." });
  }
});
```

```

    }

    try {
      const source = await prisma.watchedSource.create({
        data: { url, label, userId },
      });
      return res.status(201).json(source);
    } catch (err) {
      return res
        .status(400)
        .json({ error: "Could not create source – does that user exist?" });
    }
  });

  // LIST a user's watched sources: GET /users/:userId/sources
  app.get("/users/:userId/sources", async (req, res) => {
    const userId = Number(req.params.userId);

    if (Number.isNaN(userId)) {
      return res.status(400).json({ error: "userId must be a number" });
    }

    const sources = await prisma.watchedSource.findMany({
      where: { userId },
    });
    return res.json(sources);
  });

  // SET (create or replace) a user's profile:
  // PUT /users/:userId/profile with body { "description": "..." }
  app.put("/users/:userId/profile", async (req, res) => {
    const userId = Number(req.params.userId);
    const { description } = req.body;

    if (Number.isNaN(userId)) {
      return res.status(400).json({ error: "userId must be a number" });
    }
    if (!description || typeof description !== "string") {
      return res.status(400).json({ error: "description (string) is required" });
    }

    try {
      const profile = await prisma.profile.upsert({
        where: { userId },
        update: { description },
        create: { description, userId },
      });
      return res.json(profile);
    } catch (err) {
      return res
        .status(400)
        .json({ error: "Could not save profile – does that user exist?" });
    }
  });

  // GET a user's profile: GET /users/:userId/profile
  app.get("/users/:userId/profile", async (req, res) => {
    const userId = Number(req.params.userId);

    if (Number.isNaN(userId)) {
      return res.status(400).json({ error: "userId must be a number" });
    }

    const profile = await prisma.profile.findUnique({
      where: { userId },
    });

    if (!profile) {
      return res.status(404).json({ error: "No profile yet for this user" });
    }
    return res.json(profile);
  });

  app.listen(PORT, () => {
    console.log(`Backend listening on http://localhost:${PORT}`);
  });

  // Queue a crawl job for each of a user's watched pages.
  app.post("/users/:userId/crawl", async (req, res) => {
    const userId = Number(req.params.userId);
    if (Number.isNaN(userId)) {
      return res.status(400).json({ error: "userId must be a number" });
    }
  });

```

```

const sources = await prisma.watchedSource.findMany({ where: { userId } });
for (const source of sources) {
  await crawlQueue.add("check-source", {
    sourceId: source.id,
    url: source.url,
    userId,
  });
}

return res.json({ queued: sources.length });
});

// DELETE a watched source, and wipe its snapshots/vectors so re-adding is fresh.
app.delete("/users/:userId/sources/:sourceId", async (req, res) => {
  const userId = Number(req.params.userId);
  const sourceId = Number(req.params.sourceId);

  if (Number.isNaN(userId) || Number.isNaN(sourceId)) {
    return res.status(400).json({ error: "userId and sourceId must be numbers" });
  }

  // Find it first so we know its URL.
  const source = await prisma.watchedSource.findFirst({
    where: { id: sourceId, userId },
  });
  if (!source) {
    return res.status(404).json({ error: "Source not found" });
  }

  // Remove it from the watch list (MySQL).
  await prisma.watchedSource.delete({ where: { id: source.id } });

  // If nobody else watches this exact URL, purge its snapshots + vectors too.
  const stillUsed = await prisma.watchedSource.count({
    where: { url: source.url },
  });
  if (stillUsed === 0) {
    try {
      await fetch(`${PIPELINE_URL}/purge`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ url: source.url }),
      });
    } catch {
      // best-effort: the watch list is already updated even if purge fails
    }
  }

  return res.json({ deleted: 1 });
});

```

What each part does

- **app.use(cors())** = lets the browser dashboard (port 3000) call this API (port 4000). Browsers block cross-address calls unless the server allows it.
- **app.use(express.json())** = reads JSON bodies into req.body.
- **GET /health** = a heartbeat so tools can check the server is alive.
- **POST /users, GET /users** = create and list users.
- **POST /users/:userId/sources** = add a watched page. It rejects a duplicate URL or duplicate label for that user (returns 409 with a clear message).
- **GET /users/:userId/sources** = list a user's pages. ':userId' is a fill-in-the-blank read from req.params.
- **PUT /users/:userId/profile** = set the profile. It uses Prisma 'upsert' (update if it exists, otherwise create), so there is exactly one profile.
- **POST /users/:userId/crawl** = drop one background 'ticket' into the queue for every watched page. Returns instantly; the worker does the slow part.

- **DELETE /users/:userId/sources/:sourceId** = remove a page. It also calls the pipeline's /purge (only if no other page uses that URL) so old snapshots and vectors are wiped and re-adding the page starts fresh.

8.4 The queue and worker: redis.ts, queue.ts, worker.ts

BullMQ needs a Redis connection, a queue to hold tickets, and a worker to process them.

File: *backend/src/lib/redis.ts*

```
import IORedis from "ioredis";

const REDIS_URL = process.env.REDIS_URL ?? "redis://localhost:6379";

// BullMQ requires maxRetriesPerRequest: null on its connection.
export const connection = new IORedis(REDIS_URL, {
  maxRetriesPerRequest: null,
});
```

File: *backend/src/lib/queue.ts*

```
import { Queue } from "bullmq";

import { connection } from "../redis";

// The "crawl" queue holds tickets to check watched pages.
export const crawlQueue = new Queue("crawl", { connection });
```

File: *backend/src/worker.ts*

```
import "dotenv/config";
import { Worker } from "bullmq";

import { connection } from "../lib/redis";

const PIPELINE_URL = process.env.PIPELINE_URL ?? "http://localhost:8000";

// The worker: for each "crawl" ticket, snapshot the page via the pipeline.
const worker = new Worker(
  "crawl",
  async (job) => {
    const { sourceId, url } = job.data;
    console.log(`[worker] Checking source ${sourceId}: ${url}`);

    const res = await fetch(`${PIPELINE_URL}/snapshots`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ url }),
    });

    if (!res.ok) throw new Error(`Pipeline returned ${res.status}`);

    const data = await res.json();
    console.log(`[worker] Snapshot for ${url}: ${data.clauses_indexed} clauses indexed`);
    return data;
  },
  { connection }
);

worker.on("completed", (job) => {
  console.log(`[worker] ■ Job ${job.id} done`);
});

worker.on("failed", (job, err) => {
  console.log(`[worker] ■ Job ${job?.id} failed: ${err.message}`);
});

console.log("[worker] Waiting for crawl jobs...");
```

- **redis.ts** opens the connection (BullMQ requires maxRetriesPerRequest: null).
- **queue.ts** creates the 'crawl' queue that holds tickets.
- **worker.ts** is a separate program. For each ticket it calls the pipeline's /snapshots endpoint to fetch and index that page, and logs success or failure. The queue means nothing is lost and

failures are visible.

9. The Pipeline (Python + FastAPI + AI)

The pipeline is the brain. It fetches pages, cleans them, splits them into clauses, turns clauses into embeddings, stores them in Qdrant, computes the semantic diff, and asks Groq to summarize or explain.

9.1 MongoDB connection: db.py

File: pipeline/app/db.py

```
import os

from dotenv import load_dotenv
from pymongo import MongoClient

# Read the secrets from pipeline/.env into the program.
load_dotenv()

MONGO_URL = os.getenv("MONGO_URL", "mongodb://localhost:27017/")
MONGO_DB = os.getenv("MONGO_DB", "papertrail")

# One shared connection to MongoDB for the whole pipeline.
client = MongoClient(MONGO_URL)
db = client[MONGO_DB]

# A "collection" is like a table/folder-box. Ours holds page snapshots.
snapshots = db["snapshots"]
```

- Loads secrets from .env, opens one shared MongoDB connection, and exposes the 'snapshots' collection (Mongo's version of a table).

9.2 Clean and split: processing.py

File: pipeline/app/processing.py

```
import re

from bs4 import BeautifulSoup

def extract_text(html: str) -> str:
    """Turn a raw HTML page into clean, human-readable text."""
    soup = BeautifulSoup(html, "html.parser")

    # Throw away code-only tags that hold no readable words.
    for tag in soup(["script", "style", "noscript"]):
        tag.decompose()

    # Get the visible text, then tidy up blank lines and spacing.
    text = soup.get_text(separator="\n")
    lines = [line.strip() for line in text.splitlines()]
    return "\n".join(line for line in lines if line)

def split_into_clauses(text: str) -> list[str]:
    """Cut clean text into clauses (chunks of meaning)."""
    # Split at sentence-enders (. ! ?) and at line breaks.
    rough_pieces = re.split(r"(?<[.!?])\s+|\n+", text)

    # Keep only pieces that are long enough to carry real meaning
    # (this skips tiny bits like menu links or single words).
    clauses = [piece.strip() for piece in rough_pieces if len(piece.strip()) >= 40]
    return clauses
```

- **extract_text** uses BeautifulSoup to strip away script/style tags and pull out just the human-readable words from raw HTML.
- **split_into_clauses** uses a regular expression to cut the text at sentence-enders (. ! ?) and line breaks, and keeps only pieces at least 40 characters long (this throws away noise like menu links and single words).

9.3 The local embedding model: embeddings.py

File: pipeline/app/embeddings.py

```
from sentence_transformers import SentenceTransformer

# Load the free, local model once when the app starts.
# The FIRST time this runs, it downloads the model (~90 MB) automatically.
_model = SentenceTransformer("all-MiniLM-L6-v2")

def embed_texts(texts: list[str]) -> list[list[float]]:
    """Turn a list of texts into a list of meaning-coordinates (embeddings)."""
    vectors = _model.encode(texts, normalize_embeddings=True)
    return vectors.tolist()
```

- Loads 'all-MiniLM-L6-v2' once at startup (it downloads ~90 MB the first time).
- **embed_texts** turns a list of sentences into a list of 384-number embeddings.
- **normalize_embeddings=True** scales every pin onto a unit sphere so comparing closeness becomes a simple, clean calculation (cosine similarity).

9.4 Qdrant and the semantic diff: vectors.py

File: pipeline/app/vectors.py

```
import os
import uuid

from qdrant_client import QdrantClient
from qdrant_client.models import Distance, PointStruct, VectorParams

QDRANT_URL = os.getenv("QDRANT_URL", "http://localhost:6333")
COLLECTION = "clauses"
VECTOR_SIZE = 384 # all-MiniLM-L6-v2 produces 384 numbers per clause

client = QdrantClient(url=QDRANT_URL)

def ensure_collection() -> None:
    """Create our 'clauses' collection once, if it doesn't exist yet."""
    existing = [c.name for c in client.get_collections().collections]
    if COLLECTION not in existing:
        client.create_collection(
            collection_name=COLLECTION,
            vectors_config=VectorParams(size=VECTOR_SIZE, distance=Distance.COSINE),
        )

def index_clauses(
    url: str,
    snapshot_id: str,
    clauses: list[str],
    vectors: list[list[float]],
) -> int:
    """Store each clause + its embedding as a point in Qdrant."""
    points = [
        PointStruct(
            id=str(uuid.uuid4()),
            vector=vector,
            payload={
                "url": url,
                "snapshot_id": snapshot_id,
                "clause_index": i,
                "text": clause,
            },
        )
        for i, (clause, vector) in enumerate(zip(clauses, vectors))
    ]
    if points:
        client.upsert(collection_name=COLLECTION, points=points)
    return len(points)

# Make sure the collection exists as soon as the app starts.
ensure_collection()

from qdrant_client.models import FieldCondition, Filter, MatchValue
```

```

def _snapshot_filter(snapshot_id: str) -> Filter:
    """A filter that limits a search to one snapshot's clauses."""
    return Filter(
        must=[FieldCondition(key="snapshot_id", match=MatchValue(value=snapshot_id))]
    )

def _clause_points(snapshot_id: str):
    """Fetch every clause-point belonging to one snapshot."""
    points, _ = client.scroll(
        collection_name=COLLECTION,
        scroll_filter=_snapshot_filter(snapshot_id),
        with_vectors=True,
        with_payload=True,
        limit=10000,
    )
    return points

def _best_score(vector, snapshot_id: str) -> float:
    """Find the closeness of the nearest clause in the given snapshot."""
    result = client.query_points(
        collection_name=COLLECTION,
        query=vector,
        query_filter=_snapshot_filter(snapshot_id),
        limit=1,
    )
    return result.points[0].score if result.points else 0.0

def compute_diff(current_id: str, previous_id: str, threshold: float = 0.85):
    """Compare two snapshots and return meaningfully added/removed clauses."""
    current_points = _clause_points(current_id)
    previous_points = _clause_points(previous_id)

    # New clauses with NO close match in the old version = added/changed.
    added = []
    for p in current_points:
        score = _best_score(p.vector, previous_id)
        if score < threshold:
            added.append({"text": p.payload["text"], "closest_score": round(float(score), 3)})

    # Old clauses with NO close match in the new version = removed.
    removed = []
    for p in previous_points:
        score = _best_score(p.vector, current_id)
        if score < threshold:
            removed.append({"text": p.payload["text"], "closest_score": round(float(score), 3)})

    return added, removed

from qdrant_client.models import FilterSelector

def purge_url(url: str) -> None:
    """Delete all clause-points for a given URL from Qdrant."""
    client.delete(
        collection_name=COLLECTION,
        points_selector=FilterSelector(
            filter=Filter(
                must=[FieldCondition(key="url", match=MatchValue(value=url))]
            )
        ),
    )

```

- **ensure_collection** creates the 'clauses' collection (384 dims, cosine distance) if it does not exist.
- **index_clauses** stores each clause as a 'point': a unique id, its vector, and a payload tagging which url, which snapshot, and the clause text.
- **compute_diff** is the semantic diff. For each new clause it finds the nearest old clause; if closeness is below 0.85 it is 'added or changed'. It does the reverse for 'removed'. The threshold controls how picky it is.
- **purge_url** deletes all points for a URL (used when a page is removed).

9.5 Summaries and explanations: explain.py

File: pipeline/app/explain.py

```
import os

from groq import Groq

client = Groq(api_key=os.getenv("GROQ_API_KEY"))
MODEL = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")

def explain_changes(url: str, profile: str | None, added: list, removed: list) -> str:
    """Ask Groq to explain the change in plain English for THIS user."""
    if not added and not removed:
        return "No meaningful changes were detected."

    added_text = "\n".join(f"- {c['text']}" for c in added) or "(none)"
    removed_text = "\n".join(f"- {c['text']}" for c in removed) or "(none)"

    prompt = f"""A page the user watches has changed. Here is the change.

Who the user is: {profile or "a general user"}

Page: {url}

Clauses ADDED or CHANGED:
{added_text}

Clauses REMOVED:
{removed_text}

Explain in plain, simple English:
1. What actually changed (2-4 short bullet points).
2. How it affects THIS user specifically, given who they are.
Keep it friendly and under 150 words. Do NOT invent clauses that are not listed above."""

    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {
                "role": "system",
                "content": "You explain legal and policy fine-print changes to ordinary people in clear, simple language.",
            },
            {"role": "user", "content": prompt},
        ],
        temperature=0.3,
    )
    return response.choices[0].message.content.strip()

def summarize_clauses(clauses: list[str]) -> str:
    """Ask Groq to summarize the key points of a page's fine print."""
    if not clauses:
        return "This page has no readable fine print to summarize."

    # Cap the size so the prompt stays small.
    joined = "\n".join(f"- {c}" for c in clauses[:60])

    prompt = f"""Here are clauses from a page of fine print (terms, policy, or rules):

{joined}

Summarize the KEY points a normal person should know. as 3-6 short bullet points.
in plain simple English. Focus on money/fees, cancellation, data & privacy,
obligations, and anything risky. Keep it under 120 words."""

    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {
                "role": "system",
                "content": "You explain fine print to ordinary people in clear, simple language.",
            },
            {"role": "user", "content": prompt},
        ],
        temperature=0.3,
    )
    return response.choices[0].message.content.strip()
```

- **explain_changes** builds a prompt with the user's profile and the added/removed clauses, and asks Groq to explain what changed and how it affects THIS user. The prompt says 'do NOT invent clauses', a guard against the AI making things up.
- **summarize_clauses** asks Groq for the key points of a page in a few simple bullets.
- **temperature=0.3** keeps the AI factual and steady, not flowery.

9.6 The endpoints: main.py

This wires everything together and exposes the HTTP endpoints. FastAPI also auto-builds an interactive docs page at /docs.

File: pipeline/app/main.py

```
from datetime import datetime, timezone
from app.vectors import index_clauses, compute_diff
from app.embeddings import embed_texts
from app.processing import extract_text, split_into_clauses
import httpx
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from app.db import snapshots
from app.explain import explain_changes
from fastapi.middleware.cors import CORSMiddleware
from app.explain import explain_changes, summarize_clauses
from app.vectors import index_clauses, compute_diff, purge_url

app = FastAPI(title="PaperTrail Pipeline")

# Let the Next.js dashboard (port 3000) call this pipeline.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Describes the data we expect when someone asks us to snapshot a page.
class SnapshotRequest(BaseModel):
    url: str

def serialize(doc):
    """Turn a Mongo document into something JSON-friendly."""
    doc["id"] = str(doc.pop("_id"))
    return doc

@app.get("/health")
def health():
    return {"status": "ok", "service": "papertrail-pipeline"}

@app.get("/")
def home():
    return {"message": "PaperTrail pipeline is alive! 🟢"}

# FETCH a page and SAVE a snapshot: POST /snapshots with body { "url": "..."}
@app.post("/snapshots")
def create_snapshot(payload: SnapshotRequest):
    # 1) Fetch the page.
    try:
        response = httpx.get(payload.url, timeout=20, follow_redirects=True)
    except httpx.HTTPError:
        raise HTTPException(status_code=502, detail="Could not fetch that URL")

    # 2) Save the raw snapshot into MongoDB.
    doc = {
        "url": payload.url,
        "fetched_at": datetime.now(timezone.utc),
        "status_code": response.status_code,
        "content": response.text,
        "content length": len(response.text),
    }
    result = snapshots.insert_one(doc)
    snapshot_id = str(result.inserted_id)
```

```

# 3) Clean → split into clauses → embed → store in Qdrant.
text = extract_text(response.text)
clauses = split_into_clauses(text)
indexed = 0
if clauses:
    vectors = embed_texts(clauses)
    indexed = index_clauses(payload.url, snapshot_id, clauses, vectors)

# 4) Reply with the details.
return {
    "id": snapshot_id,
    "url": doc["url"],
    "status_code": doc["status_code"],
    "content_length": doc["content_length"],
    "clauses_indexed": indexed,
    "fetched_at": doc["fetched_at"],
}

# LIST recent snapshots (metadata only, newest first): GET /snapshots
@app.get("/snapshots")
def list_snapshots(limit: int = 10):
    # {"content": 0} means "skip the big content field" so the list stays tidy.
    cursor = snapshots.find({}, {"content": 0}).sort("fetched_at", -1).limit(limit)
    return [serialize(doc) for doc in cursor]

# DEBUG: fetch a page, clean it, and show the clauses we'd compare.
@app.post("/debug/clauses")
def debug_clauses(payload: SnapshotRequest):
    response = httpx.get(payload.url, timeout=20, follow_redirects=True)
    text = extract_text(response.text)
    clauses = split_into_clauses(text)
    return {
        "url": payload.url,
        "clause_count": len(clauses),
        "clauses": clauses[:20], # show the first 20 so the reply stays readable
    }

# A little form for comparing two sentences' meanings.
class SimilarityRequest(BaseModel):
    text_a: str
    text_b: str

# DEBUG: how similar in MEANING are two sentences? (1.0 = identical meaning)
@app.post("/debug/similarity")
def debug_similarity(payload: SimilarityRequest):
    vectors = embed_texts([payload.text_a, payload.text_b])
    a, b = vectors[0], vectors[1]

    # Because the vectors are normalized, closeness = the dot product.
    similarity = sum(x * y for x, y in zip(a, b))

    return {
        "text_a": payload.text_a,
        "text_b": payload.text_b,
        "similarity": round(similarity, 3),
        "dimensions": len(a),
    }

# --- A helper to inject test versions of a page (plain text, no fetching) ---
class IngestRequest(BaseModel):
    url: str
    text: str

@app.post("/debug/ingest")
def debug_ingest(payload: IngestRequest):
    """Store a snapshot from raw text (handy for testing the diff)."""
    doc = {
        "url": payload.url,
        "fetched_at": datetime.now(timezone.utc),
        "status_code": 200,
        "content": payload.text,
        "content_length": len(payload.text),
    }
    result = snapshots.insert_one(doc)
    snapshot_id = str(result.inserted_id)

    clauses = split_into_clauses(payload.text)
    indexed = 0

```

```

    if clauses:
        indexed = index_clauses(payload.url, snapshot_id, clauses, embed_texts(closures))

    return {"id": snapshot_id, "url": payload.url, "clauses indexed": indexed}

# --- The real semantic diff: compare a URL's two most recent snapshots ---
class DiffRequest(BaseModel):
    url: str

@app.post("/diff")
def semantic_diff(payload: DiffRequest):
    recent = list(
        snapshots.find({"url": payload.url}).sort("fetched_at", -1).limit(2)
    )
    if len(recent) < 2:
        return {
            "url": payload.url,
            "message": "Need at least two snapshots of this URL to compare.",
            "added_or_changed": [],
            "removed": [],
        }

    current_id = str(recent[0]["_id"]) # newest
    previous_id = str(recent[1]["_id"]) # the one before

    added, removed = compute_diff(current_id, previous_id)

    return {
        "url": payload.url,
        "current_snapshot": current_id,
        "previous_snapshot": previous_id,
        "added or changed": added,
        "removed": removed,
    }

class ExplainRequest(BaseModel):
    url: str
    profile: str | None = None

@app.post("/explain")
def explain(payload: ExplainRequest):
    recent = list(
        snapshots.find({"url": payload.url}).sort("fetched_at", -1).limit(2)
    )
    if len(recent) < 2:
        return {
            "url": payload.url,
            "message": "Need at least two snapshots to compare.",
            "explanation": None,
        }

    current_id = str(recent[0]["_id"])
    previous_id = str(recent[1]["_id"])
    added, removed = compute_diff(current_id, previous_id)

    explanation = explain_changes(payload.url, payload.profile, added, removed)

    return {
        "url": payload.url,
        "added or changed": added,
        "removed": removed,
        "explanation": explanation,
    }

class CheckRequest(BaseModel):
    url: str
    profile: str | None = None

@app.post("/check")
def check(payload: CheckRequest):
    # 1) Fetch the page now.
    try:
        response = httpx.get(payload.url, timeout=20, follow_redirects=True)
    except httpx.HTTPError:
        raise HTTPException(status_code=502, detail="Could not fetch that URL")

    # 2) Clean it, split into clauses, save snapshot, index in Qdrant.

```

```

text = extract_text(response.text)
clauses = split_into_clauses(text)

doc = {
    "url": payload.url,
    "fetched_at": datetime.now(timezone.utc),
    "status_code": response.status_code,
    "content": response.text,
    "content_length": len(response.text),
}
result = snapshots.insert_one(doc)
snapshot_id = str(result.inserted_id)
if clauses:
    index_clauses(payload.url, snapshot_id, clauses, embed_texts(clauses))

# 3) A summary of the page as it is right now (always useful).
summary = summarize_clauses(clauses)

# 4) How many times have we seen this page?
count = snapshots.count_documents({"url": payload.url})

# First time ever → just give the summary (nothing to compare to yet).
if count < 2:
    return {
        "status": "first",
        "summary": summary,
        "explanation": None,
        "added_or_changed": [],
        "removed": [],
    }

# 5) Compare the two most recent versions.
recent = list(
    snapshots.find({"url": payload.url}).sort("fetched_at", -1).limit(2)
)
current_id = str(recent[0]["_id"])
previous_id = str(recent[1]["_id"])
added, removed = compute_diff(current_id, previous_id)

# Nothing meaningful changed.
if not added and not removed:
    return {
        "status": "no_change",
        "summary": summary,
        "explanation": None,
        "added_or_changed": [],
        "removed": [],
    }

# Something changed → explain it for this user.
explanation = explain_changes(payload.url, payload.profile, added, removed)
return {
    "status": "changed",
    "summary": summary,
    "explanation": explanation,
    "added_or_changed": added,
    "removed": removed,
}

class PurgeRequest(BaseModel):
    url: str

@app.post("/purge")
def purge(payload: PurgeRequest):
    """Delete all snapshots + vectors for a URL (used when a page is removed)."""
    snap_result = snapshots.delete_many({"url": payload.url})
    purge_url(payload.url)
    return {"url": payload.url, "snapshots_deleted": snap_result.deleted_count}

```

The key endpoints

- **CORSMiddleware** lets the dashboard call the pipeline directly.
- **POST /snapshots** = fetch a page, save the raw HTML to Mongo, split + embed + store clauses in Qdrant.
- **POST /check** (the main one) = does a snapshot, ALWAYS makes a summary, then decides: 'first' (only summary), 'no_change', or 'changed' (summary + explanation).

- **POST /explain** and **POST /diff** = the diff and explanation used on their own.
- **POST /purge** = delete all snapshots and vectors for a URL.
- **POST /debug/*** = helper endpoints (clauses, similarity, ingest) used to test the engine with hand-made input.
- Each request body is validated by a Pydantic model (e.g. CheckRequest), so bad input is rejected automatically.

10. The Frontend (Next.js + Tailwind)

The dashboard is the face of PaperTrail: a warm 'paper' themed page where you add pages, check them, read summaries and changes, and delete pages.

10.1 Global styles: globals.css

File: frontend/src/app/globals.css

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

@media (prefers-color-scheme: dark) {
  :root {
    --background: #0a0a0a;
    --foreground: #ededed;
  }
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

button:not(:disabled) {
  cursor: pointer;
}
```

- Loads Tailwind, and adds one rule so every enabled button shows the hand pointer on hover.

10.2 The whole dashboard: page.tsx

This one file is the entire UI. It is a React 'client component' (it runs in the browser) so it can hold state and fetch data. Below is the full file, then an explanation.

File: frontend/src/app/page.tsx

```
"use client";

import { useEffect, useState } from "react";

const API_URL = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:4000";
const PIPELINE_URL =
  process.env.NEXT_PUBLIC_PIPELINE_URL ?? "http://localhost:8000";
const USER_ID = 1;

type Source = {
  id: number;
  label: string;
  url: string;
  createdAt: string;
};

type CheckResult = {
  loading?: boolean;
  error?: string;
  status?: "first" | "no_change" | "changed";
  summary?: string;
  explanation?: string | null;
};

function hostnameOf(url: string): string | null {
```

```

    try {
      return new URL(url).hostname;
    } catch {
      return null;
    }
  }
}

// Returns a clean URL string if valid, or null if it isn't a real web address.
function normalizeUrl(value: string): string | null {
  const trimmed = value.trim();
  // If they didn't type http:// or https://, assume https://
  const withProtocol = /^https?:\/\//i.test(trimmed)
    ? trimmed
    : `https://${trimmed}`;
  try {
    const u = new URL(withProtocol);
    const looksReal = u.hostname === "localhost" || u.hostname.includes(".");
    if ((u.protocol === "http:" || u.protocol === "https:") && looksReal) {
      return u.toString();
    }
  } catch {
    return null;
  }
}

function Spinner() {
  return (
    <span className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-current border-t-transparent" />
  );
}

export default function Home() {
  const [sources, setSources] = useState<Source[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [profile, setProfile] = useState<string>("");

  const [label, setLabel] = useState("");
  const [url, setUrl] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [formError, setFormError] = useState<string | null>(null);

  const [checks, setChecks] = useState<Record<number, CheckResult>>({});
  const [deleteTarget, setDeleteTarget] = useState<Source | null>(null);

  async function loadSources() {
    try {
      const res = await fetch(`${API_URL}/users/${USER_ID}/sources`);
      if (!res.ok) throw new Error("Bad response");
      setSources(await res.json());
      setError(null);
    } catch {
      setError("Could not reach the server. Is the backend running?");
    } finally {
      setLoading(false);
    }
  }

  async function loadProfile() {
    try {
      const res = await fetch(`${API_URL}/users/${USER_ID}/profile`);
      if (res.ok) {
        const data = await res.json();
        setProfile(data.description ?? "");
      }
    } catch {
      // no profile yet - fine
    }
  }

  useEffect(() => {
    loadSources();
    loadProfile();
  }, []);

  async function handleAdd(e: React.FormEvent) {
    e.preventDefault();

    if (!label.trim()) {
      setFormError("Please add a label.");
      return;
    }
  }
}

```

```

    }

    const normalized = normalizeUrl(url);
    if (!normalized) {
        setFormError("Please enter a valid URL, like https://example.com");
        return;
    }

    setSubmitting(true);
    setFormError(null);
    try {
        const res = await fetch(`${API_URL}/users/${USER_ID}/sources`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ label, url: normalized }),
        });
        if (!res.ok) {
            const data = await res.json().catch(() => ({}));
            throw new Error(data.error ?? "Could not add that page.");
        }
        setLabel("");
        setUrl("");
        await loadSources();
    } catch (err) {
        setFormError(
            err instanceof Error ? err.message : "Could not add that page."
        );
    } finally {
        setSubmitting(false);
    }
}

async function handleCheck(source: Source) {
    setChecks((c) => ({ ...c, [source.id]: { loading: true } }));
    try {
        const res = await fetch(`${PIPELINE_URL}/check`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ url: source.url, profile }),
        });
        const data = await res.json();
        setChecks((c) => ({ ...c, [source.id]: { loading: false, ...data } }));
    } catch {
        setChecks((c) => ({
            ...c,
            [source.id]: {
                loading: false,
                error: "Check failed. Are the pipeline and Docker running?",
            },
        }));
    }
}

async function confirmDelete() {
    const source = deleteTarget;
    if (!source) return;
    try {
        const res = await fetch(
            `${API_URL}/users/${USER_ID}/sources/${source.id}`,
            { method: "DELETE" },
        );
        if (!res.ok) throw new Error("Bad response");
        setSources((s) => s.filter((x) => x.id !== source.id));
        setChecks((c) => {
            const next = { ...c };
            delete next[source.id];
            return next;
        });
    } catch {
        // if it fails, we just close the box
    } finally {
        setDeleteTarget(null);
    }
}

return (
    <main className="min-h-screen bg-[#faf6ea] text-stone-800">
        </* Header */>
        <header className="border-b border-dashed border-stone-300/80 bg-[#f6efe0]/70 backdrop-blur">
            <div className="mx-auto flex max-w-4xl items-center justify-between px-6 py-4">
                <div className="flex items-center gap-3">
                    <span className="flex h-10 w-10 items-center justify-center rounded-full bg-amber-800 font-serif text-lg font-bold text-amber-50 shadow-sm">

```

```

        P
      </span>
      <div className="leading-tight">
        <div className="font-serif text-lg font-semibold tracking-tight text-stone-900">
          PaperTrail
        </div>
        <div className="text-[11px] uppercase tracking-[0.15em] text-stone-500">
          the fine print, watched
        </div>
      </div>
    </div>
    <span className="rounded-full border border-stone-300 bg-[#fffd7] px-3 py-1 text-xs font-medium
text-stone-600">
      {sources.length} on file
    </span>
  </div>
</header>

  { /* Hero */ }
  <section className="mx-auto max-w-4xl px-6 pt-14 pb-8">
    <h1 className="font-serif text-[2.6rem] leading-tight font-semibold tracking-tight text-stone-900">
      Your chance <span className="italic text-amber-700">feed</span>
    </h1>
    <p className="mt-3 max-w-xl text-[15px] leading-relaxed text-stone-600">
      Terms, policies, and rules change quietly. PaperTrail reads the fine
      print for you – and says, in plain words, what changed and how it
      touches your life.
    </p>
    <div className="mt-6 h-px w-full
bg-[repeating-linear-gradient(90deg,transparent,transparent_6px,#d6c9ad_6px,#d6c9ad_12px)]" />
  </section>

  { /* Add-a-page form */ }
  <section className="mx-auto max-w-4xl px-6 pb-10">
    <form
      onSubmit={handleAdd}
      className="rounded-lg border border-stone-300 bg-[#fffd7] p-6
shadow-[0_2px_10px_rgba(120,90,40,0.06)]"
    >
      <h2 className="font-serif text-base font-semibold text-stone-900">
        Add a page to watch
      </h2>
      <p className="mt-0.5 text-xs text-stone-500">
        Paste any link with fine print – a policy, terms, or rules page.
      </p>
      <div className="mt-4 flex flex-col gap-3 sm:flex-row">
        <input
          value={label}
          onChange={(e) => setLabel(e.target.value)}
          placeholder="Label (e.g. Netflix ToS)"
          className="flex-1 rounded-md border border-stone-300 bg-[#fbf8f0] px-4 py-2.5 text-sm
text-stone-800 outline-none transition placeholder:text-stone-400 focus:border-amber-600 focus:bg-white
focus:ring-4 focus:ring-amber-600/10"
        />
        <input
          value={url}
          onChange={(e) => setUrl(e.target.value)}
          placeholder="https://..."
          className="flex-[2] rounded-md border border-stone-300 bg-[#fbf8f0] px-4 py-2.5 text-sm
text-stone-800 outline-none transition placeholder:text-stone-400 focus:border-amber-600 focus:bg-white
focus:ring-4 focus:ring-amber-600/10"
        />
        <button
          type="submit"
          disabled={submitting}
          className="inline-flex items-center justify-center gap-2 rounded-md bg-amber-800 px-5 py-2.5
text-sm font-semibold text-amber-50 shadow-sm transition hover:bg-amber-900 disabled:cursor-not-allowed
disabled:opacity-60"
        >
          {submitting ? <Spinner /> : null}
          {submitting ? "Filing..." : "Add page"}
        </button>
      </div>
      {formError && (
        <p className="mt-3 text-sm text-red-700">{formError}</p>
      )}
    </form>
  </section>

  { /* Feed */ }
  <section className="mx-auto max-w-4xl px-6 pb-20">
    {loading && (
      <div className="flex items-center gap-2 text-sm text-stone-500">

```

```

    <Spinner /> Opening your files...
  </div>
)}

{error && (
  <div className="rounded-lg border border-red-300 bg-red-50 p-4 text-sm text-red-700">
    {error}
  </div>
)}

{!loading && !error && sources.length === 0 && (
  <div className="rounded-lg border border-dashed border-stone-300 bg-[#fffd7]/70 p-12 text-center">
    <div className="mx-auto mb-3 text-4xl">■</div>
    <p className="font-serif font-medium text-stone-700">
      Nothing on file yet
    </p>
    <p className="mt-1 text-sm text-stone-400">
      Add a page above to start watching its fine print.
    </p>
  </div>
)}

{!loading && !error && sources.length > 0 && (
  <div className="grid gap-5 sm:grid-cols-2">
    {sources.map((source) => {
      const check = checks[source.id];
      const host = hostnameOf(source.url);
      return (
        <article
          key={source.id}
          className="flex flex-col rounded-lg border border-stone-300 bg-[#fffd7] p-5
shadow-[0_2px_10px_rgba(120,90,40,0.06)] transition hover:shadow-[0_6px_20px_rgba(120,90,40,0.12)]"
        >
          <div className="flex items-start gap-3">
            <span className="mt-0.5 flex h-9 w-9 shrink-0 items-center justify-center overflow-hidden
rounded-md border border-stone-300 bg-[#fbf8f0]">
              {host ? (
                // eslint-disable-next-line @next/next/no-img-element
                <img
                  src={`https://www.google.com/s2/favicons?domain=${host}&sz=64`}
                  alt=""
                  className="h-5 w-5"
                />
              ) : (
                <span className="text-sm">■</span>
              )}
            </span>
            <div className="min-w-0 flex-1">
              <h3 className="truncate font-serif font-semibold leading-snug text-stone-900">
                {source.label}
              </h3>
              <p className="truncate text-xs text-stone-500">
                {source.url}
              </p>
            </div>
            <button
              onClick={() => setDeleteTarget(source)}
              title="Stop watching this page"
              aria-label="Delete"
              className="-mr-1 shrink-0 rounded-md p-1.5 text-stone-400 transition hover:bg-red-50
hover:text-red-600"
            >
              X
            </button>
          </div>

          <div className="mt-4 flex items-center justify-between">
            <span className="text-xs italic text-stone-400">
              Filed {new Date(source.createdAt).toLocaleDateString()}
            </span>
            <button
              onClick={() => handleCheck(source)}
              disabled={check?.loading}
              className="inline-flex items-center gap-2 rounded-md border border-amber-700/40
bg-amber-50 px-3 py-1.5 text-sm font-medium text-amber-900 transition hover:bg-amber-100
disabled:cursor-not-allowed disabled:opacity-60"
            >
              {check?.loading && <Spinner />}
              {check?.loading ? "Reading..." : "Check for changes"}
            </button>
          </div>
        </article>
      )}
    )}
  </div>
)}

{check?.error && (

```

```

    <p className="mt-4 text-sm text-red-700">{check.error}</p>
  )}

  {check && !check.loading && !check.error && (
    <div className="mt-4 space-y-3 border-t border-dashed border-stone-300 pt-4">
      {check.status === "first" && (
        <StatusPill color="stone" label="■ Key points" />
      )}
      {check.status === "no_change" && (
        <StatusPill
          color="green"
          label="■ No changes since last check"
        />
      )}
      {check.status === "changed" && (
        <StatusPill color="amber" label="■ Change detected" />
      )}

      {check.status === "channed" && check.explanation && (
        <div className="rounded-md border border-amber-300 bg-amber-50 p-3.5 text-sm
leading-relaxed text-stone-800 whitespace-pre-line">
          {check.explanation}
        </div>
      )}
    </div>

    {check.summary && (
      <div>
        <p className="mb-1.5 text-[11px] font-semibold uppercase tracking-[0.12em]
text-stone-400">
          {check.status === "changed"
            ? "Current summary"
            : "Summary"}
        </p>
        <div className="rounded-md border border-stone-200 bg-[#fbf8f0] p-3.5 text-sm
leading-relaxed text-stone-700 whitespace-pre-line">
          {check.summary}
        </div>
      </div>
    )}
  </div>
</article>
);
}}
</div>
)}
</section>

<footer className="border-t border-dashed border-stone-300/80 py-6">
  <p className="text-center font-serif text-xs italic text-stone-400">
    PaperTrail – reads the fine print so you don't have to.
  </p>
</footer>
{ /* Delete confirmation box */
{deleteTarget && (
  <div
    className="fixed inset-0 z-50 flex items-center justify-center bg-stone-900/40 p-4 backdrop-blur-sm"
    onClick={() => setDeleteTarget(null)}
  >
    <div
      className="w-full max-w-sm rounded-xl border border-stone-300 bg-[#ffdfdf] p-6 shadow-2xl"
      onClick={(e) => e.stopPropagation()}
    >
      <h3 className="font-serif text-lg font-semibold text-stone-900">
        Stop watching this page?
      </h3>
      <p className="mt-2 text-sm leading-relaxed text-stone-600">
        &ldquo;{deleteTarget.label}&rdquo; will be removed from your
        files. You can always add it again later.
      </p>
      <div className="mt-6 flex justify-end gap-3">
        <button
          onClick={() => setDeleteTarget(null)}
          className="rounded-md border border-stone-300 bg-white px-4 py-2 text-sm font-medium
text-stone-700 transition hover:bg-stone-50"
        >
          Cancel
        </button>
        <button
          onClick={confirmDelete}
          className="rounded-md bg-red-600 px-4 py-2 text-sm font-semibold text-white transition
hover:bg-red-700"
        >

```

```

        Delete
      </button>
    </div>
  </div>
</div>
)}
</main>
);
}

function StatusPill({
  color,
  label,
}): {
  color: "stone" | "green" | "amber";
  label: string;
}) {
  const styles = {
    stone: "bg-stone-200 text-stone-700",
    green: "bg-emerald-100 text-emerald-800",
    amber: "bg-amber-200 text-amber-900",
  }[color];
  return (
    <span
      className={`inline-flex items-center gap-1.5 rounded-full px-2.5 py-1 text-xs font-semibold ${styles}`}
      >
      {label}
    </span>
  );
}

```

How it works

- **'use client'** marks it as a browser component so it can use React state.
- **useState** holds: the list of sources, loading/error flags, the profile, the add-form fields, per-card check results, and the delete target.
- **useEffect(..., [])** runs once on load to fetch the sources and the profile.
- **normalizeUrl** validates the typed URL with the browser's URL parser. Junk like 'abc' is rejected; a bare 'example.com' gets 'https://' added automatically.
- **handleAdd** POSTs the new page; if the backend rejects a duplicate it shows the exact reason returned by the server.
- **handleCheck** calls the pipeline's /check and stores the result for that card.
- **confirmDelete** deletes the page after the confirmation modal.
- **StatusPill** shows a coloured label: 'Key points', 'No changes', or 'Change detected'. Explanations and summaries render with line breaks preserved.
- The delete confirmation is an in-page modal (a dim overlay with a centered card), not the browser's default popup.

11. Automation: GitHub Actions

GitHub can run tasks for free in the cloud. We use it two ways.

11.1 Continuous Integration: ci.yml

On every push it builds and type-checks all three parts, catching mistakes early.

File: *.github/workflows/ci.yml*

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  backend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - run: npm ci
      - run: npx tsc --noEmit

  frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - run: npm ci
      - run: npm run build

  pipeline:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: pipeline
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: python -m compileall app
```

11.2 The free scheduled crawl: scheduled-crawl.yml

A cron ('0 6 * * *' means 6:00 AM UTC daily) that would trigger the backend's /crawl endpoint automatically. It also has a manual 'Run workflow' button. It fully activates once the backend is deployed to a public URL (kept in a GitHub secret).

File: *.github/workflows/scheduled-crawl.yml*

```
name: Scheduled crawl

on:
  schedule:
    - cron: "0 6 * * *" # every day at 06:00 UTC
  workflow_dispatch: {} # also lets you run it manually from the Actions tab

jobs:
  crawl:
    runs-on: ubuntu-latest
```

```
steps:
  - name: Trigger the crawl for user 1
    run: |
      curl -fsS -X POST "${{ secrets.CRAWL_URL }}/users/1/crawl"
```

12. Why we did not deploy (yet)

PaperTrail is complete and fully working on a local machine. It is not deployed to the public internet yet, and that is a deliberate, reasonable choice, not a missing feature.

- **Six services, six hosts.** Deploying means putting the frontend, backend, worker, pipeline and FOUR data stores on separate free hosts, and wiring them together. That is a large, separate project on its own.
- **The AI pipeline is heavy.** The PyTorch model needs more memory than most free web hosts give (512 MB), so it needs a host like Hugging Face Spaces.
- **Free tiers sleep.** Free hosts spin down when idle, so the first request after a nap is slow (a cold start).
- **Free managed MySQL is fussy** to find compared to Postgres.

Importantly, the app is deploy-READY: environment files are templated, and the GitHub Actions cron is already written so daily crawls switch on the moment a public backend URL is set. A sensible free deployment plan would be: Frontend on Vercel; MongoDB on Atlas; Qdrant on Qdrant Cloud; Redis on Upstash; MySQL on a free host; the pipeline on Hugging Face Spaces; the backend and worker on Render.

13. Interview questions and answers

Common questions an interviewer might ask about this project, with clear answers.

Q: In one line, what does PaperTrail do?

A: It watches web pages of fine print (terms, policies, rules) and, using an AI semantic diff, tells the user in plain English what meaningfully changed and how it affects them.

Q: Why do you use BOTH MySQL and MongoDB? Isn't that overkill?

A: They store different shapes of data. MySQL holds neat, related records (users, profiles, the watch-list) where a relational schema and foreign keys shine. MongoDB holds big, unstructured raw HTML snapshots of unknown size, where a document store is a better fit. Using each store for what it is best at is called polyglot persistence.

Q: Why not just store the snapshots in MySQL too?

A: Raw HTML pages are large, unstructured blobs that vary wildly in size and shape. Forcing them into rigid relational columns is awkward, and we keep every version, so a document store that happily accepts whole documents is cleaner and scales better for this.

Q: Why do you need a vector database at all? Why not just diff the text?

A: A plain text diff screams about every reworded sentence even when the meaning is identical, and it misses meaning changes hidden behind similar words. We want MEANING changes. Embeddings turn meaning into coordinates, and a vector database (Qdrant) finds nearest meanings quickly. A normal database cannot answer 'what means almost the same?'.

Q: What is an embedding, simply?

A: A list of numbers that represents the meaning of a piece of text, like coordinates on a map of meaning. Texts with similar meaning get similar numbers, so we can measure how close two meanings are.

Q: Why run embeddings locally with sentence-transformers instead of an embedding API?

A: Cost and privacy. The local model (all-MiniLM-L6-v2) is free, runs offline, and sends no data to a third party. For a project with a hard 'everything must be free' rule, that is ideal. The trade-off is a slightly less accurate model than large paid ones.

Q: Why Groq and not OpenAI or Gemini?

A: Groq has a generous free tier and is extremely fast. We only use the LLM for the human-friendly summary and impact text, so a free, fast model is a perfect fit. The model name is configurable via an environment variable, so swapping providers is easy.

Q: Why a separate Python FastAPI service instead of doing everything in Node/Express?

A: The AI ecosystem (sentence-transformers, PyTorch) lives in Python. Keeping the ML pipeline as its own FastAPI service lets each part use the best language for the job, keeps concerns separated, and lets the pipeline scale independently of the API.

Q: Why Redis and BullMQ? What does a job queue give you?

A: Fetching and embedding a page is slow. If we did it inside the web request, the app would freeze and could time out. A queue lets us accept the work instantly, process it in the background, retry failures, and never lose a job. Redis stores the queue.

Q: Why not just run the crawl synchronously in the request?

A: It would block the request for many seconds, could hit HTTP timeouts, and would not survive a crash mid-way. A queue makes the work durable, observable and non-blocking, and it is what a real scheduled system needs.

Q: How does the semantic diff work, and what is the 0.85 threshold?

A: For each clause in the new version we ask Qdrant for the nearest clause in the old version and get a cosine-similarity score. If the best score is at least 0.85 we treat the clause as unchanged (even if reworded); below 0.85 it is genuinely new or changed. The threshold is the dial: higher is pickier (more sensitive to small edits), lower ignores more. 0.85 is a balance we can tune.

Q: What are the limitations of the diff?

A: The small free embedding model sometimes rates two paraphrases as less similar than a human would, causing a few false alarms. It can be improved with a larger local model, a tuned threshold, or by grouping related clauses before comparing.

Q: Why Prisma, and why did you pin it to version 6?

A: Prisma gives a clean, type-safe way to define tables and query them without writing raw SQL. We pinned to v6 because v7 introduced a breaking change (it no longer allows the database URL directly in the schema and requires driver adapters), which added complexity we did not need for this project. Pinning locks the version so a surprise update cannot break the build.

Q: Why Docker Compose?

A: It runs all four databases with one command, identically on any machine, with no manual installs, and keeps data in named volumes. It makes the project reproducible and free.

Q: What is CORS and why did you need it?

A: CORS (Cross-Origin Resource Sharing) is a browser safety rule that blocks a page from one address calling a server at a different address unless the server allows it. Our dashboard (port 3000) calls the backend (4000) and the pipeline (8000), so both enable CORS to permit it.

Q: Why a monorepo?

A: The three parts are developed together and share context. One repository keeps them versioned together, simplifies CI, and makes the project easy to clone and run as a whole, while each part still lives in its own folder.

Q: How does the 'summary first, changes later' logic work?

A: The /check endpoint counts how many snapshots exist for a URL. On the very first check there is nothing to compare to, so it returns just a summary. On later checks it runs the diff: if nothing meaningful changed it says so, otherwise it returns the change explanation plus the current summary.

Q: What happens when a user deletes a page? Why is that non-trivial?

A: Data lives in three stores. Deleting only from MySQL would leave old snapshots in Mongo and vectors in Qdrant, so re-adding the page would falsely report a change against stale data. So on delete we also call the pipeline's /purge to wipe the snapshots and vectors, but only if no other watched page still uses that URL.

Q: How do you prevent duplicates?

A: Before creating a source, the backend checks whether that user already has the same URL or the same label, and returns a 409 with a specific message if so. MySQL's default collation also makes the label check case-insensitive.

Q: How would you scale this?

A: Run multiple queue workers to process crawls in parallel, cache summaries in Redis, batch embeddings, add a Qdrant payload index on the url field, and shard MongoDB by url. The queue-and-worker design already supports horizontal scaling of the heavy work.

Q: What are the security considerations?

A: Secrets (database passwords, the Groq key) live only in .env files that are git-ignored. For production I would add authentication (there is a single demo user today), rate limiting, input sanitisation, restricted CORS origins, and least-privilege database users instead of root.

Q: Why isn't it deployed, and what would deploying involve?

A: It is a large separate effort: six services on six free hosts, plus the heavy PyTorch pipeline needing more RAM than typical free web tiers. The app is deploy-ready (templated env files, a GitHub Actions cron) and a concrete free plan exists (Vercel, Atlas, Qdrant Cloud, Upstash, a free MySQL host, Hugging Face Spaces, Render).

Q: What is the role of GitHub Actions here?

A: Two roles: Continuous Integration (build and type-check the code on every push to catch errors early) and a free scheduled cron that will trigger the daily crawl once the backend is public.

Q: Why the 40-character filter when splitting clauses?

A: It removes noise such as menu links, single words and copyright lines, so we only embed and compare real, meaning-carrying sentences. That keeps the diff focused and cheaper.

Q: Why normalize embeddings, and what is cosine similarity?

A: Normalizing puts every vector on a unit sphere so their length does not matter, only their direction. Cosine similarity measures the angle between two vectors: 1.0 means the same direction (same meaning), 0 means unrelated. Normalizing makes this comparison clean and fast.

Q: Why Next.js App Router and a client component for the dashboard?

A: Next.js gives a fast React setup with great tooling and easy free hosting on Vercel. The dashboard is interactive (state, fetching, forms), so it is a client component; server components cannot hold interactive state.

Q: How does the worker talk to the pipeline?

A: The worker (Node) makes an HTTP POST to the pipeline's /snapshots endpoint using the PIPELINE_URL from its environment. The two services communicate over HTTP, which also means they can live on different hosts after deployment.

14. Limitations and future improvements

- Add real user accounts and authentication (today there is one demo user).
- Deploy all services so the scheduled cron catches changes automatically.
- Use a larger or fine-tuned embedding model, or tune the 0.85 threshold, to reduce false alarms on paraphrases.
- Store detected changes as a history so the feed shows a timeline, not just the latest.
- Send notifications (email) when an important change is detected.
- Cache summaries and add a Qdrant payload index for speed at scale.
- Add tests for the diff logic and the API routes.

PaperTrail - reads the fine print so you don't have to.